

ZAP! Known Limitations & Workarounds

The author of this software stands in solidarity with 🇺🇦 Ukraine. We believe in a world where international borders are respected and human rights are upheld. We encourage all users of this software to contribute to humanitarian efforts in 🇺🇦 Ukraine.

A quick-reference guide to what ZAP! intentionally does not support and how to work around each constraint.

Version: 1.0 **Date:** March 2026

Architecture & Platform

6502/65C02 Target Only

ZAP! compiles exclusively to 6502/65C02 assembly (via ca65/ld65). There is no support for other CPUs or platforms.

16-Bit Address Space

All pointers are 2 bytes (WORD). The maximum addressable memory is 64 KB (\$0000-\$FFFF).

256-Byte Zero-Page Limit

The 6502 zero-page (\$00-\$FF) is shared between system temporaries (MATH_STACK, TMP0-TMP5) and user variables. Programs with many pointers or small variables can exhaust it. Use `-cfg <path>` to automatically read the ZP budget from your ld65 linker config, or `-ZPSTART <addr>` to set it manually (e.g., `-ZPSTART 0x82` for Atari 8-bit). The compiler detects ZP overflow at compile time and reports a clear error.

Workaround: Reduce pointer count; use WORD variables instead of extra pointers where possible:

```
; Instead of many pointers:
byte ^ptr1, ^ptr2, ^ptr3      ; 6 ZP bytes

; Store addresses in words and load into one pointer when needed:
byte ^ptr
word addr1, addr2, addr3      ; 6 ZP bytes but only 2 for the pointer
```

All Types Are Unsigned

ZAP! has no signed integer types. BYTE is 0-255, WORD is 0-65535, LONG is 0-4294967295.

Workaround: Implement signed logic manually. For example, treat values ≥ 128 as negative for BYTE:

```
func byte is_negative(byte val)
    if val >= 128
```

```

        return 1
    end
    return 0
end

```

No Floating-Point

There are no float or double types. All arithmetic is integer.

Workaround: Use fixed-point arithmetic with lookup tables:

```

; Fixed-point 8.8: high byte = integer, low byte = fraction
word fixed_value = $0180      ; 1.5 in 8.8 format

; Or use pre-computed lookup tables
byte sine_table[] = {0, 13, 26, 38, 50, 61, 71, 78, 83, 86, 88}

```

No Dynamic Memory

ZAP! does not support `malloc/free` or any heap allocation. All memory is statically allocated at compile time.

Workaround: Use fixed-size arrays as object pools:

```

const byte MAX_SPRITES = 10
byte sprite_x[MAX_SPRITES]
byte sprite_y[MAX_SPRITES]
byte sprite_active[MAX_SPRITES]
byte sprite_count = 0

proc spawn_sprite(byte x, byte y)
    if sprite_count < MAX_SPRITES
        sprite_x[sprite_count] = x
        sprite_y[sprite_count] = y
        sprite_active[sprite_count] = 1
        sprite_count = sprite_count + 1
    end
end

```

No True Recursion

Parameters and local variables are stored in fixed memory locations (zero-page or BSS), not on a call stack. A recursive call overwrites the caller's locals.

Workaround: Rewrite recursive algorithms iteratively, using explicit arrays as a manual stack:

```
; Instead of recursive tree traversal:
byte stack[32]
byte sp = 0

proc push(byte val)
    stack[sp] = val
    sp = sp + 1
end

func byte pop()
    sp = sp - 1
    return stack[sp]
end
```

Type System

Enum Base Types: BYTE and WORD Only

Enums cannot use LONG as a base type.

```
enum byte Color          ; OK
    RED, GREEN, BLUE
end

enum word BigEnum        ; OK
    LARGE_VAL = 1000
end

; enum long X ... end    ; ERROR: unsupported base type
```

Single-Character Literals Only

Character literals produce a BYTE value. Multi-character literals are not supported.

```
byte c = 'A'              ; OK: c = 65
; byte c = 'AB'           ; ERROR
```

Enum Qualification: Dot Notation Only

When two enums share a member name, use `EnumName.Member`. Colon syntax (`EnumName:Member`) is not supported.

```
byte val = Color.RED      ; OK
```

```
; byte val = Color:RED    ; ERROR
```

Variables & Declarations

Declarations Must Precede Statements

Inside a procedure or function body, all local variable declarations must appear before the first executable statement.

```
proc valid()
    byte x = 10          ; declarations first
    byte y = 20
    x = x + y            ; then statements
end

; proc invalid()
;     byte x = 10
;     x = x + 1          ; statement
;     byte y = 20        ; ERROR: declaration after statement
; end
```

CONST Restrictions

- Must have an initializer (`const byte X` alone is invalid)
- Cannot have a fixed address (`const byte X @$2000` is invalid)
- `sizeof()` in const expressions requires a **type name**, not a variable: `const word SZ = sizeof(Point)` works, but `const word SZ = sizeof(p)` does not (use the type name instead)

STATIC Restrictions

- Only allowed on **local** variables (not globals)
- Must have an initializer
- Cannot be combined with CONST
- Cannot be combined with PORT

```
proc counter()
    static byte count = 0    ; OK
    count = count + 1
end
```

PORT Restrictions

- Requires a fixed address (`@$xxxx`)
- Cannot be used on arrays or pointers
- Cannot have an initializer

- **#RD** and **#WR** modifiers are only valid together with **#PORT**

```
byte GTIA_HPOS0 @$D000 #PORT #WR    ; OK: write-only port
; byte arr[10] @$D000 #PORT          ; ERROR: port cannot be array
```

Array Size Must Be Compile-Time Constant

No variable-length arrays. Array dimensions must be positive integer constants.

```
const byte SIZE = 10
byte data[SIZE]          ; OK
; byte data[some_variable] ; ERROR
```

No Scalar Array Initialization

You cannot initialize an entire array to a single value in the declaration.

```
byte arr[10]                ; OK: uninitialized (BSS-zeroed)
byte arr[] = {1, 2, 3}      ; OK: explicit initializer list
; byte arr[10] = 0          ; ERROR: scalar init not supported
```

Workaround: Use a loop to fill the array:

```
byte arr[10]
byte i
proc fill_array()
    for i = 0 to 10
        arr[i] = 0
    end
end
```

Control Flow

FOR Loops: Positive Step Only

The **step** value must be a positive compile-time constant. Negative steps are not supported.

Workaround: Use a **while** loop to count down:

```
; Cannot do: for i = 10 to 0 step -1

byte i
proc count_down()
```

```

    i = 10
    while i > 0
        ; process i = 10, 9, 8, ... 1
        i = i - 1
    end
end

```

FOR Loop Bound Is Exclusive

for i = 0 to N iterates from 0 through N-1 (N iterations total). The upper bound is **not** included.

```

byte i
proc example()
    for i = 0 to 5
        ; i takes values: 0, 1, 2, 3, 4 (NOT 5)
    end
end

```

BREAK and CONTINUE Target Innermost Loop Only

There is no labeled break or multi-level break. **break** exits only the immediately enclosing loop or switch. **continue** skips to the next iteration of the immediately enclosing loop.

Workaround: Use a flag variable:

```

byte done = 0
byte i, j

proc nested_search()
    for i = 0 to 10
        for j = 0 to 10
            if found_it(i, j)
                done = 1
                break                ; exits inner loop only
            end
        end
        if done
            break                ; exits outer loop
        end
    end
end

```

SWITCH: Case Labels Must Be Constants

Case labels must be compile-time constant expressions. Variables or function calls are not allowed.

```
const byte STATE_PLAY = 1
byte state = STATE_PLAY

proc update()
  switch state
    case STATE_PLAY          ; OK: constant
      ; ...
      break
    case 2                   ; OK: literal
      ; ...
      break
    ; case some_var          ; ERROR: not a constant
  end
end
```

SWITCH: No Duplicate Case Labels

Each case value must be unique within the same switch statement.

Expressions

Expression Nesting Depth Limited

The compiler uses an 8-slot math stack for evaluating complex expressions. Extremely deeply nested expressions (more than ~8 levels of binary operators) will fail with "Math stack overflow."

Workaround: Break complex expressions into temporary variables:

```
; Instead of one huge expression:
; result = a + b * (c + d * (e + f * (g + h * (i + j))))

; Use temps:
byte t1 = i + j
byte t2 = h * t1
byte t3 = g + t2
; ... and so on
```

No Pointer Addition

Adding two pointers is not allowed (`ptr1 + ptr2`). You can add a pointer and an integer, or subtract two pointers of the same type.

No Struct Arithmetic

A struct variable holds a group of fields, not a single number, so the compiler cannot add, subtract, multiply, or compare two structs as a whole. Any binary operator (`+`, `-`, `*`, `/`, `%`, `==`, `!=`, `<`, `>`, `<=`, `>=`, `&&`, `||`) applied directly to a struct value is rejected at compile time.

This restriction applies **only to whole struct values**. Individual struct fields behave like ordinary scalars, and pointers to structs behave like ordinary pointers:

```

struct Point
    byte x
    byte y
end

Point p1
Point p2
Point ptr a, b

; NOT allowed – whole struct values:
; byte sum = p1 + p2          ; ERROR: struct in arithmetic
; if p1 == p2 then ... end if ; ERROR: struct in comparison

; Allowed – scalar fields:
byte sum = p1.x + p2.x        ; OK: BYTE + BYTE
if p1.x == p2.x then          ; OK: comparing scalars
    ; ...
end if

; Allowed – struct pointers:
if a == b then ... end if     ; OK: pointer comparison
a = a + 1                     ; OK: pointer arithmetic

```

Workaround: If you need to "compare" or "combine" two structs, operate on their fields one by one, or write a helper PROC that does the field-wise work for you.

No Bitwise Operations on Pointers

Bitwise operators (&, |, ^, ~, <<, >>) cannot be applied to pointer types.

Workaround: Cast the pointer to a WORD first if you truly need bit manipulation on an address.

Address-Of (@) Limited to Simple LValues

The @ operator works with simple variables, array elements with simple indices, and struct fields. Complex computed expressions are not supported.

```

byte arr[10]
byte i = 5
word addr = @arr[i]          ; OK: simple index
; word addr = @arr[i + 1]    ; ERROR: complex subscript

```

Workaround: Store the index in a variable first:

```
byte idx = i + 1
word addr = @arr[idx]      ; OK
```

LOW/HIGH: Not for Structs

`low()` and `high()` work on scalar and pointer types only. Passing a struct value is an error.

LOWW/HIGHW: LONG Only

`loww()` and `highw()` accept only LONG-type expressions. Passing BYTE or WORD is an error.

Pointers

Pointers Must Reside in Zero-Page

The 6502's indirect addressing mode (`addr`), `Y` requires the pointer to be in zero-page. The compiler places all pointers in ZP. If zero-page is exhausted, pointer allocation fails.

No Pointer-to-Pointer Chains

You cannot create a pointer that points to another pointer. Assigning one pointer to another copies the *address value*, not a reference to the pointer itself.

```
byte x = 100
byte ^ptr1 = @x
byte ^ptr2 = ptr1      ; ptr2 gets the same address as ptr1 (copy)
                        ; NOT a pointer to ptr1
```

Pointer Comparison Restricted

Pointers can only be compared against other pointers of the same type or the literal constant `0`. Comparing a pointer to a BYTE variable or arbitrary integer is rejected.

```
byte ^ptr = @data
if ptr != 0      ; OK: compare to NULL/0
    ; ...
end
; if ptr > 100    ; ERROR: cannot compare pointer to arbitrary integer
```

Port Dereference Checks: Simple Identifiers Only

The `#RD`/`#WR` access checks work only when dereferencing a simple pointer identifier (`ptr^`). Computed pointer expressions (`(expr)^`) bypass the check.

Structs & Arrays

No Whole Struct-Array Copy

Assigning one struct array to another (`dst = src` where both are arrays of structs) is not supported.

Workaround: Copy element by element in a loop:

```
struct Point
  byte x
  byte y
end

Point src[5]
Point dst[5]
byte i

proc copy_points()
  for i = 0 to 5
    dst[i] = src[i]      ; single struct copy IS supported
  end
end
```

Local Structs Always Considered Initialized

The definite-assignment analysis does not track per-field initialization for struct variables. Local structs are always treated as initialized, which means reading uninitialized fields is not caught at compile time.

Best practice: Always initialize struct fields explicitly after declaration.

Local Arrays Always Considered Initialized

Similarly, local arrays bypass uninitialized-variable detection. Array elements in BSS are zero on first call but retain values on subsequent calls to the same procedure.

No Runtime Array Bounds Checking

The compiler does not insert bounds checks at runtime. Accessing an out-of-bounds index silently reads/writes incorrect memory.

Best practice: Always validate indices against array size before access.

Struct Field Access on Function Call Re-Evaluates

Accessing different fields of a function return value causes the function to be called multiple times:

```
; Each field access calls make_point() separately:
byte x = make_point(10, 20).x    ; call 1
byte y = make_point(10, 20).y    ; call 2
```

Workaround: Store the result in a temporary struct:

```
Point p = make_point(10, 20)    ; single call
byte x = p.x
byte y = p.y
```

Functions & Procedures

No Function Overloading

Each function or procedure name must be unique. You cannot define two functions with the same name but different parameter lists.

No Variadic Parameters

Functions and procedures accept a fixed number of parameters. There is no `...` or variable-argument mechanism.

Parameters Are Pass-By-Value

All parameters (including structs) are copied when passed. Modifying a parameter inside a procedure does not affect the caller's variable.

Workaround: Pass a pointer to modify the original:

```
proc set_value(byte ^ptr, byte val)
    ptr^ = val
end

proc main()
    byte x = 0
    set_value(@x, 42)    ; x is now 42
end
```

FUNC Must Have Top-Level RETURN

A function must have a `return` statement reachable at the function's top level. Having `return` only inside `if/else` branches is not sufficient.

```
; This works:
func byte min(byte a, byte b)
    if a < b
        return a
    end
    return b                ; top-level return required
end
```

```
; This does NOT work:
; func byte min(byte a, byte b)
;     if a < b
;         return a
;     else
;         return b          ; ERROR: no top-level return
;     end
; end
```

Return Type Validation

The compiler validates that **return** expressions match the declared return type:

- **Scalar widening/narrowing** is allowed freely (BYTE ↔ WORD ↔ LONG)
- **Pointers are WORD-compatible** — returning a pointer where WORD is expected (or vice versa) is allowed
- **Struct returns require exact type match** — returning a different struct type, or a scalar where a struct is expected, is a compile-time error
- **Missing return expression** in a **func** is a compile-time error

```
func Point make()          ; Returns struct
    return 42              ; ERROR: expected struct 'POINT', got BYTE
end

func byte get_val()
    Point p
    return p               ; ERROR: expected BYTE, got struct 'POINT'
end
```

Default Parameters Must Follow Required Parameters

Parameters with default values must come after all required parameters.

```
proc draw(byte x, byte y, byte color = 1)    ; OK
; proc draw(byte x = 0, byte y)              ; ERROR
```

Module System

No Circular Includes

If module A includes module B and module B includes module A, compilation fails.

Workaround: Extract shared definitions into a third module that both A and B include.

Module Files Cannot Define main()

The `main()` entry point must be in the top-level program file, not in a module.

Module Name Must Be Quoted

The `.module` directive requires a quoted string:

```
.module "math_lib"      ; OK
; .module math_lib      ; ERROR
```

Inline Assembly

Directives Only Inside `asm...end` Blocks

Assembler directives like `.segment`, `.incbin`, `.byte` are only recognized inside `asm ... end` blocks. They cannot appear at the ZAP top level.

```
proc load_data() #keep
    asm
        .segment "DATA"
        .incbin "sprite.dat"
    end
end
```

Notes

- The compiler automatically restores the CODE segment after each `asm ... end` block, so manual `.segment "CODE"` is not needed.
- ZAP compiler does not check anything inside an `asm` block — the content is copied verbatim into the generated assembly.
- If you need assembly only to include data (as shown above), use `#keep` to prevent the procedure from being optimized away when it is not called.
- If you happen to call `load_data()`, nothing catastrophic happens — the compiler emits only `RTS` in the `CODE` segment at the end of the procedure, and the data resides in the `DATA` segment. However, if you place raw data into the `CODE` segment and then call the procedure, the CPU will attempt to execute the data as instructions, with unpredictable results.
- ZAP language is case-insensitive, but the ca65 assembler is not. Remember this when referring to ZAP variables from an `asm` block.

Do Not Use `END` as an Assembly Label

The keyword `END` terminates the `ASM` block. Using it as a label or instruction operand prematurely closes the block.

Calling Convention Is Not Stable

Parameter passing is optimized per compilation. Register assignments (A, X, Y) for procedure parameters may change. Avoid calling ZAP procedures with more than 3 bytes of parameters from inline assembly.

ASM Blocks Are Opaque to the Optimizer

The peephole optimizer and dead-code eliminator do not analyze or transform code inside `asm ... end` blocks.

Identifiers

Leading Underscore Forbidden

Identifiers starting with `_` are reserved for compiler-generated symbols. Source code cannot use them.

```
; byte _count = 0      ; ERROR: leading underscore reserved
byte count = 0         ; OK
```

Case-Insensitive

ZAP! identifiers are case-insensitive. `myVar`, `MyVar`, and `MYVAR` all refer to the same symbol.

Source File Encoding

ASCII Only

ZAP! source files must contain only ASCII characters (0-127). Unicode characters (including common symbols like `→`, `—`, `≥`, accented letters, emoji, etc.) in comments, string literals, or identifiers will cause a compilation error:

```
UnicodeEncodeError: 'charmap' codec can't encode character ...
```

This is because the generated assembly is processed by the ca65 assembler which uses platform-specific character encoding. Use ASCII equivalents instead:

```
; byte val → result    ; ERROR: Unicode arrow
; byte val -> result    ; OK: ASCII arrow
```

String literals support escape sequences for special characters: `\n`, `\t`, `\xHH`, `\\`, `\"`, etc. See the Language Reference for the full list.

Uninitialized Variable Detection Gaps

The definite-assignment analysis catches most uninitialized reads but has known blind spots:

- **Inter-procedural initialization:** If a procedure writes through a pointer, the caller still sees the variable as uninitialized.
- **Pointer aliasing:** `ptr^ = 42` does not mark the pointed-to variable as initialized.
- **WHILE loop bodies:** Variables initialized only inside a WHILE may be flagged since the loop might not execute.
- **Globals:** Only local variables are tracked; globals are not analyzed.

See [Safety Features](#) in the Advanced Topics for full details.

For complete language syntax and semantics, see the [ZAP! Language Reference](#). For advanced techniques and workarounds, see [Advanced Topics](#).